InsightFlow-Agent2 · 项目说明

一句话：把"每周看数据、写结论"这件重复劳动，变成"丢进去两张表、拿到一份带图、 可追溯、能自动推到群里的报告"。

技术栈：Python 3.11 · LangGraph · FastAPI · PostgreSQL + pgvector · Redis · Docker · pytest **仓库**：github.com/shauige66-hash/InsightFlow-Agent2

一、解决什么问题

谁用：1–5 人小团队里兼做数据的那个人（运营 / 客服主管），不是专业分析师。

原来怎么做：导出 CSV → Excel 透视 → 挑数 → 手写结论，每周约 90 分钟。 只来得及看总量；结论说不出出处；遇到脏数据（月份不完整、运费被当成商品）会得出 错误结论且不自知。

改成什么：上传表 → 提固定的几个问题 → 拿到带图、带口径、带来源的报告，跑完 自动推飞书。

不适用：单次、小数据、临时问一句——那用 ChatGPT 更快。这套的价值在**重复**。

二、做法

一条 23 个节点的 LangGraph 流水线：数据 → 上下文 → 知识检索 → 规划 → 计划校验 → 〔审批〕→ 路由 → 执行（容器内）→ 审查 → 决策 → 推进 → 可视化 → 报告 → 产物。

1. 治理收敛到唯一入口。 模型调用与工具执行都经过 `harness/`，预算、账本、幂等 只实现一次。预算按**真实上游请求**计数并前置于请求：上限 3 次就恰好 3 次。曾发现 一个真实缺陷——节点只返回部分 state 让累计用量回退，上限 3 实际发了 4 次。

2. 计划不信任模型自评。 程序校验任务结构、ID 唯一性、目标表是否属于本次 Run 的 授权资源、来源类型能否落到单一工具、依赖是否成环；不合法携带原因重写，最多 3 次。 审批记录计划指纹，执行前比对。

3. 动态决策有边界。 执行完先构造观察，模型在 7 个动作里选一个，再由确定性策略 校验：权限/授权/环境/预算类失败**禁止重试**；决策步数、重规划、补检索、重复动作各有 硬上限。

4. 结论可追溯、可被口径约束。 业务口径写成可检索的知识文档而非提示词；报告附 来源附录（文档、页码、chunk、相似度）与图表；口径变化即失效旧证据。

三、证据

工程

指标	数值
自动化用例	**1158 通过 / 2 跳过**（74 文件，约 27 分钟）
执行侧上下文	5 表场景单任务 4236 → 847 字符（**−80%**）
预算准确性	上限 N 次 → 恰好 N 次真实请求（用例断言）
崩溃恢复	3 个窗口跨进程强杀 + PostgreSQL 重连
数据隔离	令牌 → 归属者，越权读他人会话返回 404

真实模型（经 ModelGate → DeepSeek）

场景	结果
小数据（8 行订单 / 5 客户）	completed，**3/3 图表**，18 次请求，100 秒
零售案例（17,049 行 / 1,990 SKU / 177 客户）	completed，**3/3 图表**，26 次请求，95 秒

对照实验：口径的价值（同一份数据、同一个问题，两次真实运行）

	无口径	有口径	标准答案（脚本算）
月份判断	"11、12 月两个完整月"	"10、11 月是唯一可比月份，12 月截止 12-09"	12 月只有 9 天
GMV 环比	**−74.44%**	**+1.98%**	+2.0%

	无口径	有口径	标准答案（脚本算）
退货冠军	`M`（手工调整）	`22423`	`22423`（809.70）

两次工程链路都正常（3 张图、completed、报告结构完整），**错的是业务常识。**把口径写成 知识文档随分析传入，同一个模型给出正确答案。这是"系统"相对"直接问模型"的差别：口径 可被检索、审阅、按租户替换。

四、局限

1. 评测用脚本化模型验证工程链路，真实模型的结论只在小样本上验证过。
2. 认证是"令牌 → 归属者"的简单映射，没有注册、配额、账单。
3. 沙箱依赖宿主 Docker；Docker Desktop 上建议本机直接跑。
4. 队列只支持 PostgreSQL，`claimed` 超时回收需人工 `requeue`，无自动重试。
5. 单机部署，未验证水平扩展。

五、怎么验证

```
python scripts/check_model.py                              # 凭据是否可用（401/402/404）
python scripts/real_model_smoke.py                         # 真实模型跑一次完整分析
python scripts/real_model_regression.py --include-retail # 固定用例回归，带历史对比
```